pytest + Playwright Python interview questions, answered with this framework

These are pytest, Playwright for Python, API testing and Python questions that come up in SDET and test automation interviews. Each answer has a short explanation and, where useful, an example. Each one ends with an **In the framework** pointer: the file in this repository that does it, or the reason the framework does not use it. Links go to github.com/niranjankagri/python-pytest-framework.

Versions: Python 3.12, pytest 9.1, pytest-xdist 3.8, pytest-html 4.2, Playwright 1.63, pytest-playwright 0.10, requests 2.34, pydantic 2.14, ruff 0.17.

Contents

1. pytest fundamentals (Q1-Q14)
2. Fixtures in depth (Q15-Q22)
3. Plugins, parallel runs and hooks (Q23-Q29)
4. Playwright for Python (Q30-Q43)
5. API testing with requests and pydantic (Q44-Q54)
6. Python for test automation (Q55-Q62)
7. Framework design (Q63-Q69)
8. Flaky tests, debugging and reporting (Q70-Q75)
9. CI/CD (Q76-Q80)
10. Scenario questions (Q81-Q86)
11. Quick reference

1. pytest fundamentals

Q1. Why pytest rather than unittest?

	unittest	pytest
Test shape	class extending `TestCase`	plain function (classes are optional)
Assertions	`self.assertEqual(a, b)` and others	plain `assert a == b`, with a detailed diff on failure
Set-up	`setUp` / `tearDown` per class	fixtures: reusable, composable, with scopes
Data-driven	loops or subTest	`@pytest.mark.parametrize`, one report row per case
Extensions	few	1,500+ plugins (xdist, html, playwright, ...) and hooks

pytest also runs existing unittest tests, so a team can move over gradually.

In the framework: every test is a plain function; for example tests/api/test_notes.py.

Q2. How does pytest find tests?

pytest walks the `testpaths` directories, or the current directory if none are set. It collects:

- files named `test_*.py` or `*_test.py`;
- inside them, functions named `test*`;
- `test*` methods of classes named `Test*` that have no `__init__`.

The patterns can be changed with `python_files`, `python_functions` and `python_classes`.

In the framework: `testpaths = tests` in pytest.ini.

Q3. How can a plain `assert` show such detailed failures?

pytest rewrites the `assert` statements of test modules (and conftest files) at import time. The rewritten code records the value of each sub-expression, so a failure shows something like `assert 401 == 200` with `where 401 = <Response [401]>.status_code`. A second argument adds a message: `assert response.status_code == 200, response.text`.

Rewriting only applies to test modules and conftest files. Asserts in helper modules work, but they show less detail unless the module is registered with `pytest.register_assert_rewrite`.

In the framework: messages show what the API answered, for example the demo test test_report_demo.py and its report row in the README.

Q4. What is `conftest.py`?

`conftest.py` is a file that pytest loads automatically. Its fixtures and hooks are available to every test in its directory and below, without any import. A project can have several of them; one closer to a test can override a fixture from one higher up.

In the framework: a single tests/conftest.py holds the settings, API clients, throwaway accounts, pages and report hooks.

Q5. What is parametrize, and why give the cases ids?

`@pytest.mark.parametrize` runs one test function once per data set. Each case is its own test, with its own pass/fail and report row, so one bad case does not hide the others. `ids` gives each case a readable name, which you can then select with `-k`:

```
@pytest.mark.parametrize(
    ("field", "value", "message"),
    [("email", "not-an-email", "A valid email address is required"), ...],
    ids=["invalid-email", "name-too-short", "password-too-short"],
)
def test_register_validates_its_fields(api_client, unregistered_cleanup, field, value, message): ...
```

`pytest -k password-too-short` runs one case. Stacking two parametrize decorators gives every combination of their values.

In the framework: tests/api/test_users.py and `test_create_note_in_each_category` in test_notes.py.

Q6. What are markers? Why use `--strict-markers`?

Markers are labels on tests (`@pytest.mark.api`). You can then select tests by label: `-m "ui or e2e"`, `-m "api and not smoke"`. Some markers have a built-in meaning, for example `skip`, `skipif`, `xfail`, `parametrize` and `usefixtures`.

A typo such as `@pytest.mark.smok` normally creates a new, unknown marker with only a warning, and the test then silently drops out of `-m smoke`. With `--strict-markers`, any marker that is not declared fails the run.

In the framework: `api`, `ui`, `e2e`, `smoke` and `demo` are declared in pytest.ini. A module-level `pytestmark = pytest.mark.api` marks every test in a file.

Q7. skip, skipif, xfail: what is the difference?

- `pytest.skip("reason")` / `@pytest.mark.skip`: the test does not run. Use it when the test cannot run, for example because a feature is switched off.
- `@pytest.mark.skipif(condition, reason=...)`: skips the test only when the condition is true, for example `sys.platform == "win32"`.
- `@pytest.mark.xfail(reason=...)`: the test runs, and a failure is expected. It is used for a known bug: the run stays green, and the report shows `xfailed`.
- `xfail(strict=True)`: if the test unexpectedly passes (XPASS), the run fails. This tells you the bug is fixed and the mark should go. Without `strict`, the xfail would quietly stay forever.

In the framework: one skip and one strict xfail in tests/demo/test_report_demo.py.

Q8. What does `pytest.ini` hold, and how does `addopts` work?

`pytest.ini` holds the project's pytest configuration: `testpaths`, `markers`, and `addopts`. `addopts` holds options that are added to every run, as if typed on the command line. The same settings can instead go in `pyproject.toml` (`[tool.pytest.ini_options]`) or `tox.ini`.

Command-line options come after `addopts`, so for most options the command line wins. For example, a second `-m` replaces the `-m "not demo"` from `addopts`, which is how `pytest -m demo` works.

In the framework: pytest.ini sets `-ra --strict-markers -m "not demo"`, the HTML report, and Playwright's failure artefacts.

Q9. How do you select which tests run?

Command	Runs
`pytest tests/api`	one directory
`pytest tests/api/test_notes.py::test_get_note_by_id`	one test (node id)
`pytest -k "search and not without"`	tests whose name matches the expression
`pytest -m smoke`	tests with a marker
`pytest --lf` / `--ff`	only the last failures / the last failures first
`pytest --sw`	stepwise: stop at the first failure, and resume from it next time
`pytest -x` / `--maxfail=3`	stop after the first failure / after 3 failures

Q10. What does `-ra` do?

`-r` adds a short summary at the end of the console output. `a` means every result except passes: failed, error, skipped, xfailed and xpassed, each with its reason. In CI logs this is the quickest way to see why something did not pass.

In the framework: in `addopts`.

Q11. How do you test that code raises an exception?

```
with pytest.raises(ValueError, match="must be positive"):
    parse_amount("-1")
```

`match` is a regular expression that is searched for in `str(exception)`. Note that an API test does not test an exception; it tests a status code. The client returns the response, and the test asserts on its status code.

In the framework: not needed, because the API client never raises on 4xx (see Q45).

Q12. What is the difference between a failure and an error in pytest?

- A **failure** means an assertion or an exception inside the test function itself.
- An **error** means something went wrong in a fixture's set-up or teardown.

If `account` cannot register its user, the test reports as `ERROR`, not `FAILED`. You can see straight away that the test never got to check anything.

In the framework: fixtures assert with clear messages, for example `"Could not register a test user: ..."` in conftest.py.

Q13. What are the Arrange-Act-Assert and one-behaviour-per-test rules?

Each test is set up (arrange), does one thing (act), and checks the outcome (assert). Blank lines separate the three parts. A test name describes the behaviour, for example `test_logout_invalidates_the_token`, so a failing name in the report already says what broke.

In the framework: every test follows this shape, for example `test_patch_changes_only_the_completed_flag`.

Q14. How do you compare whole objects or several fields at once?

Compare tuples, sets or models in a single assert, so that a failure shows all the differences at once:

```
assert (note.title, note.description, note.category) == (data.title, data.description, category)
assert {note.id for note in notes} == created          # order does not matter
assert ApiResponse[Note].model_validate(response.json()).data == note   # pydantic models compare by
value
```

In the framework: test_notes.py.

2. Fixtures in depth

Q15. What is a fixture?

A fixture is a function marked with `@pytest.fixture` that provides something to a test: data, a client, a logged-in page. A test asks for a fixture by naming it as a parameter. Fixtures can ask for other fixtures, and pytest builds the chain in the right order:

```
notes_page ← app_page ← page (pytest-playwright) ← context ← browser
           ← logged_in_client ← account ← settings
```

In the framework: conftest.py.

Q16. How does teardown work in a fixture?

The code before `yield` is the set-up. The yielded value goes to the test. The code after `yield` is the teardown, and it runs even when the test failed:

```
@pytest.fixture
def account(settings):
    credentials = new_user()
    ...register...
    yield registered
    _delete_account(settings, registered.email, registered.password)
```

If the set-up itself fails before `yield`, the teardown does not run, because there is nothing to clean up. `request.addfinalizer(fn)` is the older way to register a teardown.

Q17. What are fixture scopes?

Scope	Created once per	Example
`function` (default)	test	`account`, `page`
`class`	test class	
`module`	file	
`package`	package	
`session`	run (per xdist worker)	`settings`, `browser`

A fixture can only use fixtures of the same or a wider scope. A session fixture cannot use a function-scoped one.

In the framework: `settings` is session-scoped because it never changes; accounts are function-scoped so that every test is isolated.

Q18. How do you override a fixture that a plugin provides?

Define a fixture with the same name in `conftest.py`. If it asks for itself as a parameter, it gets the original value, which it can extend:

```
@pytest.fixture(scope="session")
def browser_context_args(browser_context_args):
    return {**browser_context_args, "viewport": {"width": 1366, "height": 900}}
```

In the framework: exactly this, in conftest.py, to give every browser context a desktop-sized viewport.

Q19. What is a "factory as fixture"?

Here the fixture yields something the test can call or fill in, instead of a single object. The test then decides how many items it needs, and the fixture cleans up all of them:

```
@pytest.fixture
def unregistered_cleanup(settings):
    created = []
    yield created                       # the test appends (email, password)
    for email, password in created:
        _delete_account(settings, email, password)
```

In the framework: `unregistered_cleanup` is used by the register tests, which create users themselves.

Q20. What are `autouse` and `usefixtures`?

- `@pytest.fixture(autouse=True)` applies a fixture to every test in its scope without the test asking for it. Use it sparingly, because it hides dependencies.
- `@pytest.mark.usefixtures("name")` applies a fixture whose value the test does not need.

In the framework: neither is used. Every test names exactly what it needs, so its signature documents its set-up.

Q21. Which built-in fixtures do you know?

- `tmp_path`: a fresh temporary directory per test.
- `monkeypatch`: temporarily sets environment variables or attributes, undone after the test.
- `capsys` / `caplog`: capture printed output or log records.
- `request`: information about the requesting test (`request.node`, `request.param`), plus `addfinalizer`.
- `pytestconfig`: access to the options and the ini values.

Q22. How do you pass data into a fixture?

There are two ways:

- **Indirect parametrization**: `@pytest.mark.parametrize("account", [...], indirect=True)`. The fixture reads the value with `request.param`.
- **A parametrized fixture**: `@pytest.fixture(params=[...])`, which runs every test that uses it once per value.

In the framework: not needed, because the data comes from factories (Q66).

3. Plugins, parallel runs and hooks

Q23. How does pytest-xdist run tests in parallel?

`pytest -n 4` starts 4 worker processes. The main process collects the tests and hands them out to the workers. Points that interviewers check:

- Session fixtures run **once per worker**, not once per run.
- Tests must not share state: no shared user, no order dependency, no fixed file names.
- `--dist loadscope` keeps the tests of one module or class on the same worker, and `--dist loadfile` does the same per file. The default, `load`, sends each test to any free worker.
- `-n auto` uses one worker per CPU core.

In the framework: `-n 4` for the API tests and `-n 3` per browser in CI. This is safe because every test registers its own user (factories.py).

Q24. Which plugins does this framework use, and what for?

Plugin	For
pytest-xdist	parallel runs
pytest-html (+ pytest-metadata)	HTML report and its environment table
pytest-playwright	the `browser`, `context` and `page` fixtures, and the `--browser`, `--headed`, `--screenshot`, `--video` and `--tracing` options

Others that are often asked about:

- **pytest-rerunfailures** (`--reruns 2`) re-runs failed tests;
- **pytest-timeout** fails a test that hangs;
- **pytest-cov** measures coverage;
- **allure-pytest** produces Allure reports;
- **pytest-bdd** runs Gherkin features.

Q25. What are hooks? Give an example.

Hooks are functions with fixed names (`pytest_*`) that pytest calls at points in its lifecycle. You implement them in `conftest.py` or in a plugin. Some common ones:

- `pytest_configure`: at start-up;
- `pytest_collection_modifyitems`: to reorder or mark tests after collection;
- `pytest_runtest_makereport`: builds each test's report;
- `pytest_sessionfinish`: at the end of the run.

In the framework: in conftest.py:

- `pytest_runtest_makereport` (a hookwrapper): when a UI test fails, it takes a screenshot and adds it to the HTML report.

- `pytest_configure`: adds the app and API URLs to the report's environment table.
- `pytest_html_report_title`: sets the report title.

Q26. What is a `hookwrapper`?

A hookwrapper is a hook implementation that wraps all the other implementations of the same hook. It `yield`s, and the yield returns the outcome of the others, which it can then read or change:

```
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        ...add a screenshot to report.extras...
```

`report.when` is `"setup"`, `"call"` or `"teardown"`. The screenshot is wanted only for a failure in the test body, and `item.funcargs` gives access to the test's `page` fixture.

Q27. How do you access a fixture's value in a hook?

`item.funcargs` is a dict of the fixture values the test received, so `item.funcargs.get("page")` returns the Playwright page if the test used one. Check `page.is_closed()` first, and wrap the screenshot in `try/except`. A crashed page must not hide the real failure with a second error.

Q28. How do you add custom command-line options?

Use `pytest_addoption(parser)` in the root `conftest.py`, then read the value with `request.config.getoption("--env")`:

```
def pytest_addoption(parser):
    parser.addoption("--env", default="demo")
```

In the framework: environment variables are used instead (Q60). They work the same in a local shell, in IDE run configurations and in CI, without extra options.

Q29. How do you see `print` output, or log, during a run?

pytest captures stdout/stderr and logs, and shows them only for failed tests. Use `-s` to see output live, `--log-cli-level=INFO` to stream log records, and `caplog` to assert on log records in a test.

4. Playwright for Python

Q30. Playwright or Selenium?

	Selenium	Playwright
Protocol	W3C WebDriver (HTTP)	one persistent connection (CDP for Chromium; its own protocols for Firefox and WebKit)
Waiting	explicit waits written by you	auto-wait before every action, plus web-first assertions
Browsers	Chrome, Firefox, Edge, Safari (real browsers)	Chromium, Firefox, WebKit builds (also branded Chrome/Edge)
Isolation	one driver per browser start	many cheap browser contexts in one browser
Built-in	WebDriver only	network interception, tracing, video, codegen, API request context
Languages	Java, Python, C#, JS, Ruby	TS/JS, Python, Java, .NET

Selenium is the better choice where real Safari, a Selenium Grid or an existing Java Selenium codebase matters. Playwright usually gives faster and less flaky web tests.

Q31. Browser, context, page: what are they?

- **Browser**: one browser process, which is expensive to start. pytest-playwright starts it once per session (per worker).
- **Context**: an isolated profile, like a fresh incognito window. Each context has its own cookies, localStorage and cache. It is cheap to create, so each test gets a new one.
- **Page**: a tab inside a context.

So tests are isolated from each other without restarting the browser.

In the framework: the `page` fixture of pytest-playwright gives a new context and page per test.

Q32. What is auto-waiting?

Before an action, Playwright waits until the element is ready for it. For example, `click()` waits until the element is attached, visible, stable (not animating), enabled, and receives events (not covered by another element). Most hand-written waits from Selenium are therefore not needed. A timeout error appears only if the element never becomes ready.

In the framework: no `sleep` anywhere. The action timeout is set by `page.set_default_timeout(settings.ui_timeout_ms)` in `app_page`.

Q33. What are web-first assertions (`expect`)?

`expect(locator).to_have_text("...")` re-checks the condition until it is true or the timeout (5 s by default) ends. An `assert locator.inner_text() == "..."` checks only once, at a moment when the page may still be updating. Use `expect` for anything shown in the page.

```
expect(notes_page.progress).to_have_text("You have completed all notes")
expect(notes_page.card_titles).to_have_text([notes["Work"].title])   # the exact list of elements
```

In the framework: UI tests assert with `expect`; plain `assert` is used only for API values.

Q34. Which locators do you prefer, and why?

In order of preference:

1. **User-facing locators**: `get_by_role("button", name="Save")`, `get_by_label`, `get_by_text`. They follow what the user sees, and they also check accessibility.
2. **Test ids**: `get_by_test_id("note-submit")`. These are stable, made for tests, and unaffected by text or style changes.
3. **CSS**, with XPath as a last resort. These are tied to the DOM structure.

Locators are lazy: they are resolved at each use. A locator does not go "stale" the way a Selenium `WebElement` does.

In the framework: the app has `data-testid` attributes, so the page objects use `get_by_test_id` (notes_page.py).

Q35. How do you find one item in a list, for example one card?

Chain and filter locators:

```
self.cards.filter(has=self.page.get_by_test_id("note-card-title").get_by_text(title, exact=True))
```

This finds the card whose title element has exactly that text. Actions on child elements are then scoped to that card: `card(title).get_by_test_id("note-delete").click()`.

In the framework: `NotesPage.card()`.

Q36. How do you wait for "either A or B"?

Use `locator.or_(other)`. Add `filter(visible=True)` when both elements are always in the DOM and only one of them is visible:

```
self.progress.or_(self.no_notes_message).filter(visible=True).first.wait_for()
```

In the framework: `NotesPage.wait_until_loaded()`. The progress element is in the page but hidden when there are no notes. The first version waited for it and timed out on an empty list.

Q37. Strict mode: what happens when a locator matches several elements?

Actions such as `click` and `fill`, and single-element assertions, fail with a "strict mode violation" if the locator matches more than one element. This is deliberate: clicking "the first one" by accident hides wrong locators. Narrow the locator, or say what you want with `.first`, `.nth(i)` or `filter()`.

Q38. How do you log in once instead of in every test?

There are two common ways:

- `storage_state`: log in once and save the cookies and localStorage to a file, then create every context with `storage_state="state.json"`.
- **Inject the session yourself**: get a token through the API and put it where the app keeps it.

In the framework: the second way. The `notes_page` fixture calls `context.add_init_script("window.localStorage.setItem('token', '...')")` before the first navigation. Each test logs in its own user through the API, and the browser starts out logged in. The login form itself is tested once, in test_login.py.

Q39. How do you intercept or mock network requests?

Use `page.route(pattern, handler)` or `context.route(...)`. In the handler, call `route.abort()`, `route.continue_()` (optionally with changed headers or body) or `route.fulfill(status=..., body=...)` to return a fake response. This is useful to:

- block third-party content;
- simulate a server error;
- test a UI state that is hard to reach.

`page.expect_response(...)` waits for a real response and returns it.

In the framework: `app_page` blocks the demo site's ad and analytics hosts with `context.route("**/*", _block_ads)`.

Q40. How do you debug a failing Playwright test?

- **Trace Viewer**: `--tracing=retain-on-failure`, then `playwright show-trace test-results/<test>/trace.zip`. You get a timeline of every action, with DOM snapshots before and after, network requests, the console, and the source line.
- **Headed run, slowed down**: `--headed --slowmo 500`.
- **Inspector**: `PWDEBUG=1 pytest ...` steps through the test, or call `page.pause()` in the code.
- **Screenshot and video** of the failure.

In the framework: the trace, video and screenshot are kept for every failed UI test (`addopts`), and the HTML report embeds a screenshot.

Q41. How do you run on several browsers?

Use `--browser chromium|firefox|webkit`, which can be repeated. `--browser-channel chrome` or `msedge` uses the installed branded browser, and `--device "iPhone 15"` emulates a phone. In CI, a matrix job per browser runs the browsers in parallel and shows separate results.

In the framework: the `ui` job in ci.yml is a Chromium/Firefox/WebKit matrix.

Q42. Sync or async API?

Playwright for Python has both. pytest-playwright uses the **sync** API, which is simpler to read and fine because each test drives one page. The async API (`async_playwright`, `await page.click()`) suits asyncio applications, or one process driving many pages at once.

Q43. How do you handle iframes, new tabs and dialogs?

- **iframes**: `page.frame_locator("#frame").get_by_role(...)`.
- **New tab**: `with context.expect_page() as info: link.click()`, then `new = info.value`.
- **Dialogs** (`alert` / `confirm`): `page.on("dialog", lambda d: d.accept())`. Playwright dismisses dialogs automatically if there is no handler.

- **File upload**: `locator.set_input_files(path)`.
- **Download**: `with page.expect_download() as d: ...`.

In the framework: the app's delete confirmation is an HTML modal, not a browser dialog, so it is an ordinary locator (`note-delete-confirm`).

5. API testing with requests and pydantic

Q44. Why use a `requests.Session`?

A session reuses the TCP/TLS connection between calls (connection pooling), which is noticeably faster over HTTPS. It can also hold default headers and cookies. Always set a `timeout`: without one, a hung server blocks the test forever.

In the framework: each `NotesApiClient` holds one session and passes `timeout=settings.api_timeout` on every call (notes_client.py).

Q45. Should the API client assert on status codes, or call `raise_for_status()`?

No. The client returns the response, and the test decides what is correct. Negative tests are half of API testing: 400, 401, 404 and 409 are the expected results there. A client that raised on 4xx would need a second client for those tests.

In the framework: every client method returns `requests.Response`.

Q46. `data=` or `json=`?

- `data={...}` sends `application/x-www-form-urlencoded`, which is form fields.
- `json={...}` serialises to JSON and sends `application/json`.

Use what the API documents. Form data is text, so booleans must be sent as the strings the API expects.

In the framework: the Notes API takes form data, so `completed` is sent as `"true"` / `"false"` (`update_note`, `set_completed`).

Q47. What does a contract (schema) test check, and how does pydantic do it?

A contract test checks that the response has the agreed shape: every field present, with the right types and allowed values, and no surprise fields. In pydantic you describe the shape once as a model:

```
class Note(Contract):            # Contract sets extra="forbid"
    id: str
    category: Literal["Home", "Work", "Personal"]
    completed: bool
    created_at: datetime
```

`ApiResponse[Note].model_validate(response.json())` raises a `ValidationError` that lists every mismatch. The test then works with typed objects (`note.completed`) instead of `body["data"]["completed"]`.

In the framework: models.py.

Q48. Why `extra="forbid"`?

By default, pydantic ignores fields it does not know. A renamed field (`completed` → `is_completed`) would then show up only as a "missing field". With `extra="forbid"`, every added or renamed field fails a test straight away, so a contract change is noticed and agreed on, never silent.

Q49. How do you model an envelope that wraps different payloads?

Use a generic model:

```
class ApiResponse(Contract, Generic[T]):
    success: bool
    status: int
    message: str
    data: T | None = None

ApiResponse[Note]          # one note
ApiResponse[list[Note]]    # a list
ApiResponse[User]          # a user
```

Q50. pydantic or JSON Schema (jsonschema library)?

Both check the shape. pydantic models are Python code: typed and editor-friendly, and they give you objects to work with afterwards. JSON Schema is language-neutral, so it suits sharing the contract with other teams, or generating it from an OpenAPI file. For consumer-driven contracts between services, Pact goes a step further: the consumer's tests produce the contract, and the provider verifies it.

Q51. PUT or PATCH: how do you test the difference?

PUT replaces the whole resource, and PATCH changes only the fields that are sent. Test exactly that:

- after a PUT, every field has its new value;
- after a PATCH of one field, the other fields are unchanged.

In the framework: `test_put_replaces_every_field` and `test_patch_changes_only_the_completed_flag`.

Q52. Which negative and security checks belong in API tests?

- Validation: missing or too-short fields, wrong formats. Check the status **and** the message.
- Authentication: no token, an invalid token, and a token after logout.
- Authorisation: user B must not read or delete user A's data. This is the most common API security bug (BOLA/IDOR, number 1 in the OWASP API Top 10).
- Error leaks: a wrong password and an unknown e-mail should get the **same** answer, so that an attacker cannot find out which e-mails exist.
- Bad ids: a malformed id (400) is not the same as an unknown id (404).

In the framework: all of these, in test_users.py and test_notes.py (`test_users_cannot_see_or_delete_each_others_notes`).

Q53. How do you handle authentication in the client?

The client logs in, keeps the token, and adds the header in one place (`_request`). Tests never build headers themselves. To test a missing or wrong token, a test can set `client.token = None` or `client.token = "invalid"`.

In the framework: `NotesApiClient.login` and `_request`; `test_profile_requires_a_valid_token`.

Q54. Can Playwright test APIs too?

Yes. `playwright.request.new_context(base_url=...)` returns an `APIRequestContext`, and `page.request` shares cookies with the browser. It is handy when the API calls need the browser's session. requests is the common choice when there is no browser involved.

In the framework: requests, because the API tests run without a browser in their own CI job.

6. Python for test automation

Q55. What is a dataclass, and why make it `frozen`?

`@dataclass` generates `__init__`, `__repr__` and `__eq__` from the type-annotated fields. `frozen=True` makes instances read-only, and therefore hashable. That suits values that must not change, such as settings and test data.

In the framework: `Settings`, `Credentials` and `NoteData` are frozen. `Account` in conftest is deliberately **not** frozen, because a password-change test updates it so that the teardown can still log in.

Q56. What do `yield` and generators have to do with fixtures?

A function with `yield` is a generator: it pauses at `yield` and resumes when asked. pytest runs a fixture up to `yield` (set-up), runs the test, and then resumes the generator (teardown). It is the same idea as a context manager (`with ...:`), and `contextlib.contextmanager` uses it too.

Q57. Which type hints do you use, and why in tests?

- Built-in generics: `list[str]`, `dict[str, int]`.
- Unions: `str | None`.
- `Literal["Home", "Work", "Personal"]`.
- `Iterator[T]` for yield fixtures.
- `Generic[T]` with a `TypeVar`.

Type hints document the code, make editor completion work, and in pydantic they **are** the validation rules.

Q58. Comprehensions and set operations: an example from tests?

```
created = {create(client, "Home").id, create(client, "Work").id}
assert {note.id for note in notes} == created
notes = {category: create_via_api(client, category) for category in ("Home", "Work", "Personal")}
```

Q59. What are `*args` and `**kwargs`?

`*args` collects extra positional arguments into a tuple, and `**kwargs` collects extra keyword arguments into a dict. A thin wrapper uses them to pass options through:

```
def _request(self, method, path, **kwargs):
    headers = kwargs.pop("headers", {})
    ...
    return self._session.request(method, url, headers=headers, timeout=self._timeout, **kwargs)
```

Q60. How do you handle configuration and secrets?

- Environment variables, read in one module, with defaults for local runs.
- Secrets never go in code or in the repository; they come from CI secrets or a vault. `.env` files stay out of git.

In the framework: config.py. The public demo needs no secrets, because every test creates its own user.

Q61. Virtual environments and pinned dependencies: why?

`python -m venv .venv` keeps the project's packages separate from the system and from other projects. Exact pins (`pytest==9.1.1`) make every machine and CI run use the same versions, so a new release cannot break the suite overnight. Upgrades happen on purpose, in a commit of their own.

In the framework: requirements.txt.

Q62. What do you lint with?

ruff does both linting and formatting in one fast tool. It replaces flake8, isort, pyupgrade and black. The rule sets used here are:

- pycodestyle (E, W) and pyflakes (F);
- isort (I);
- bugbear (B): likely bugs;
- pyupgrade (UP);
- flake8-pytest-style (PT);
- simplify (SIM).

CI runs `ruff check` and `ruff format --check` before any test.

In the framework: pyproject.toml.

7. Framework design

Q63. How is the framework layered?

```
tests      what is checked        tests/api, tests/ui, tests/e2e
fixtures   set-up and clean-up    tests/conftest.py
framework  how to talk to the app framework/api, framework/pages, framework/data
config     where the app is       framework/config.py
```

Tests depend only on the layer below them. If an endpoint or a locator changes, one place changes.

In the framework: see the README and framework-design.md.

Q64. Should page objects contain assertions?

Usually not. A page object describes the page (locators) and what a user can do on it (actions). The test states the expectation. That keeps page objects reusable for positive and negative tests, and a failure points at the test, not at the framework. Waiting for the result of an action is fine inside the page object: `add_note` waits for the new card, and `delete_note` waits for the card to be detached.

In the framework: notes_page.py.

Q65. How do you keep tests independent?

- Every test creates its own data and removes it in teardown.
- No test relies on another test's result, or on the order of tests.
- There is no shared mutable state: no global variables, no shared user.

The proof: the suite passes with `-n 4`, in any order.

In the framework: the `account`, `second_account` and `unregistered_cleanup` fixtures.

Q66. How do you generate test data?

Use factory functions that return unique, valid data on every call (a `uuid` suffix), with parameters for the parts a test cares about (`new_note(category="Work")`). Libraries such as Faker give realistic names and addresses. Here uniqueness matters more than realism.

In the framework: factories.py.

Q67. What is a hybrid (API + UI) test, and why write one?

A hybrid test prepares the state through the API, which is fast and reliable, and uses the browser only for the behaviour under test. It can then check the result through the other channel. For example, complete a note in the UI, then check `completed == true` through the API. This makes the test faster and less flaky, and it checks that the front end and the back end agree.

In the framework: tests/e2e/test_api_and_ui.py.

Q68. How would you organise the tests of a bigger product?

- One package per area (`tests/api/notes`, `tests/ui/checkout`) and one client or page object per service or page.
- Shared fixtures go in the root `conftest.py`, and area-specific ones go in the area's `conftest.py`.
- Use markers for the type of test and for the cadence (`smoke` on every push, the full suite nightly).
- Keep the framework code an installable package if several repositories use it.

Q69. When should you not automate a test?

- One-off checks.
- Features that are still changing every week.
- Checks that need human judgement: visual appeal, usability.
- Anything cheaper to cover at a lower level. A validation rule belongs in an API or unit test, not in a browser test.

8. Flaky tests, debugging and reporting

Q70. What makes UI tests flaky, and how do you prevent it?

Cause	Prevention
Timing (the page is still updating)	auto-waiting actions, `expect` instead of `assert`, no `sleep`
Shared or leftover data	a fresh user per test, clean-up in teardown
Order dependency	independent tests, run in parallel to prove it
Third-party content (ads, pop-ups)	block it with `route`
Fragile locators	test ids or roles, not layout-based CSS/XPath
Environment (slow site)	timeouts in config, nightly runs to spot trends

In the framework: each of these is done (Q32-Q39, Q65).

Q71. Should you re-run failed tests automatically?

Re-runs (`pytest-rerunfailures --reruns 2`) hide flakiness as often as they help. If you use them, report the re-runs, and treat a test that needs a re-run as a bug to fix. A known product bug is better handled with `xfail(strict=True)`, linked to the ticket.

In the framework: no re-runs. The suite is stable, and the demo shows strict xfail.

Q72. What goes into a good failure report?

A good failure report shows what was expected and what was found (the assertion), plus where and when (the environment and URLs). It also includes the evidence needed to fix it without running it again: a screenshot, a trace and the API response message.

In the framework: a screenshot embedded in the HTML report row, and a trace and video in `test-results/`. The assertion messages include the API's answer, and the report's environment table shows the app and API URLs.

Q73. pytest-html or Allure?

- **pytest-html** gives one self-contained HTML file with nothing extra to install. It is easy to keep as an artifact or to send to someone.
- **Allure** gives richer reports: steps, history, trends and categories. It needs the Allure CLI or a server to build them.

In the framework: pytest-html. Playwright traces already give step-by-step detail.

Q74. How do you debug a test that fails only in CI?

1. Download the artifacts: the HTML report and the trace from the failed job.
2. Open the trace and compare it with a local run.
3. Typical causes are a different screen size (set the viewport), a headless-only behaviour, a slower machine (timeouts), missing fonts or locales, and parallel runs that share data.
4. Reproduce locally with the CI settings: headless, the same browser, the same `-n`.

Q75. How do you get to a debugger in pytest?

- `pytest --pdb` opens pdb at the point of failure.
- `breakpoint()` in a test stops there.
- `--trace` stops at the start of each test.
- For Playwright, use `PWDEBUG=1` or `page.pause()`.

9. CI/CD

Q76. Describe the pipeline.

```
lint (ruff) ──▶ api (-n 4)
            └─▶ ui matrix: chromium | firefox | webkit (-n 3 each)
```

The pipeline runs on push and on pull requests. It also runs nightly by cron, because the public site can change on its own, and on demand with a choice of markers. Each job writes a pass/fail table to the run summary and uploads its report and traces.

In the framework: ci.yml, summary.py.

Q77. Why `fail-fast: false` in a matrix?

By default, a failure in one matrix job cancels the others. With `fail-fast: false`, all browsers finish. You can then see whether a failure is specific to Firefox or happens everywhere.

Q78. How do you keep a pipeline fast?

- Lint first, so cheap failures stop the run before expensive ones.
- Run jobs in parallel (API and the browser matrix), and run tests in parallel within a job (xdist).
- Install only the browser each job needs.
- Cache dependencies (`setup-python` with `cache: pip`).
- Run smoke tests on every push and the full suite nightly, once the suite is large.

Q79. What does `if: always()` do on a step?

The step runs even when an earlier step failed. Reports are needed most when the tests failed, so the summary and artifact steps use it.

Q80. How do results get to people?

- The run's summary page, with a table built from the JUnit XML.
- Artifacts: the HTML report and the traces.
- Required status checks on pull requests.
- Optionally, a notification (Slack or Teams) on a failed nightly run.

10. Scenario questions

Q81. A UI test fails once in twenty runs. What do you do?

1. Look at the failed run's trace: what was on the screen and in the network at that moment.
2. Typical findings:
   - an assertion with `assert` that should be `expect`;
   - a click before the list re-rendered;
   - data left over by another test;
   - an ad covering the element.
3. Fix the cause. Do not add a sleep or a re-run.
4. Repeat the test many times to confirm, for example `pytest --count=50` (pytest-repeat) or a loop, with `-n` for speed.

Q82. The API changed a field name. How would this framework show it?

The pydantic models use `extra="forbid"`, so every test that parses that response fails with a `ValidationError`: "Extra inputs are not permitted" for the new name, and "Field required" for the old one. It is fixed in one place, `models.py`, after confirming that the change was intended.

Q83. How would you test 50 combinations of form input?

- Use parametrize with ids, and keep the cases in a list or a CSV/JSON file.
- Test most combinations at the API level, which is fast, and keep only a few in the UI, to check that the UI shows the errors.

In the framework: validation is tested through the API (`test_register_validates_its_fields`); the UI checks the messages of an empty form once.

Q84. The test environment is shared and other people change data. How do you stay stable?

- Never depend on existing data. Create your own, with unique names, and delete it afterwards.
- Assert only on your own data. For example, check the set of ids you created, not "the list has 3 items".
- If possible, give the test runs their own environment or tenant.

In the framework: this is exactly the situation on the public demo site.

Q85. How do you prove that a user cannot access someone else's data?

1. Create two users.
2. User A creates a resource.
3. User B tries to read, change and delete it, and gets an error each time: 404 here, or 403.
4. User A still has the resource unchanged.

In the framework: `test_users_cannot_see_or_delete_each_others_notes`, using `account` and `second_account`.

Q86. How would you add visual or accessibility checks?

- **Visual**: Playwright's `expect(page).to_have_screenshot()` is part of the TypeScript test runner. In Python, use a snapshot plugin (for example pytest-playwright-visual) or a service such as Applitools or Percy. Mask dynamic areas, and run in a fixed container so fonts render the same way.
- **Accessibility**: inject axe-core (for example the axe-playwright-python package) and fail on serious violations. Using role-based locators also helps.

In the framework: not included; this is the next extension point.

11. Quick reference

```
pytest                                    all tests (addopts: -ra, strict markers, not demo, HTML
report)
pytest -n 4                               4 parallel workers
pytest -m api | -m "ui or e2e" | -m smoke select by marker
pytest -k search                          select by name
pytest --browser firefox --browser webkit run UI tests on several browsers
pytest --headed --slowmo 500              watch the browser
pytest --lf | --ff | -x | --sw            last failed | failed first | stop at first | stepwise
pytest --pdb                              debugger at the failure
pytest -m demo                            report demo: 2 failed, 1 skipped, 1 xfailed
playwright show-trace test-results/<test>/trace.zip
playwright codegen https://practice.expandtesting.com/notes/app
ruff check . && ruff format --check .
```

Concept	Where
Fixtures, teardown, override, factory fixture	`tests/conftest.py`
Report hooks (screenshot, environment)	`tests/conftest.py`
API client	`framework/api/notes_client.py`
Contracts	`framework/api/models.py`
Page objects	`framework/pages/`
Test data	`framework/data/factories.py`
Config	`framework/config.py`
Markers, addopts	`pytest.ini`
CI	`.github/workflows/ci.yml`